VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI - 590 018



SUBJECT: DIGITAL IMAGE PROCESSING

SUBJECT CODE: BEC602

Assignment on

"Image Transform"

Submitted by

Madeeha Sakeena

1AY23EC064

Under the guidance of

Dr. Jayalaxmi H.

Associate Professor,

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

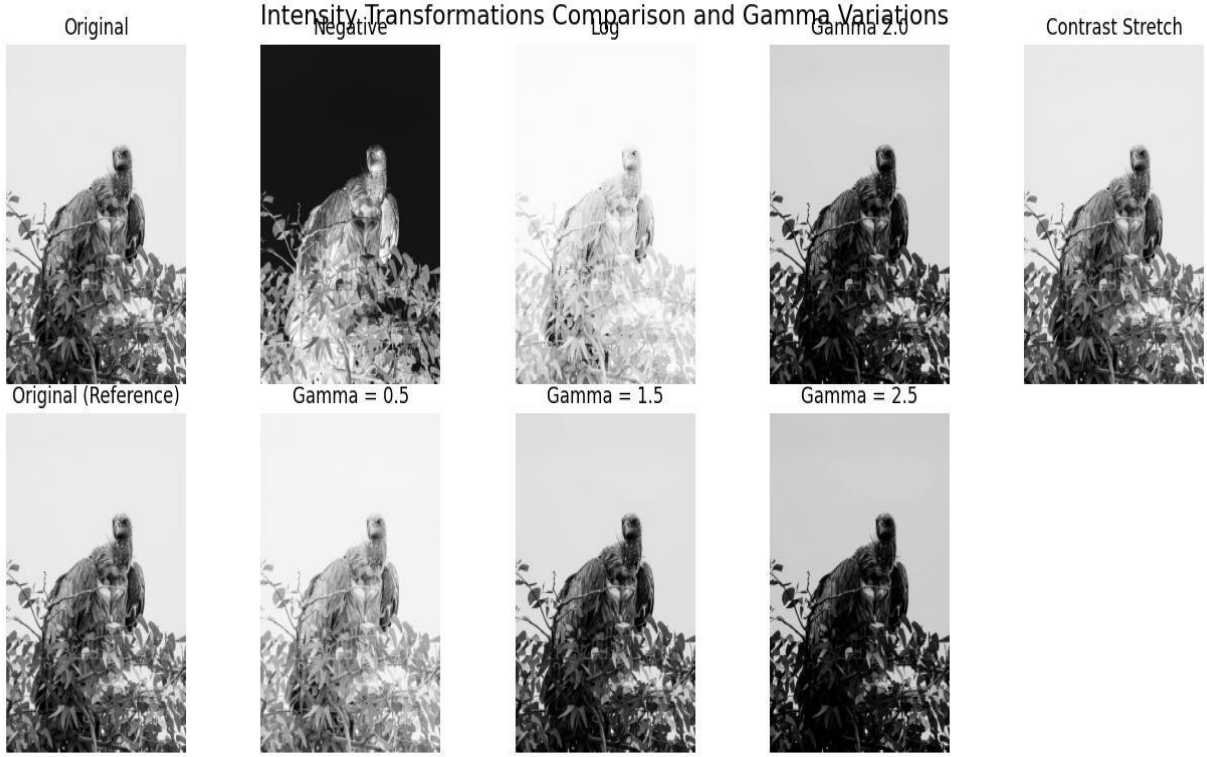
Acharya Institute of Technology



ACHARYA

Assignment 1: Intensity Transformation Functions

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def perform_analysis (image_path):
img = cv2.imread(image_path, cv2. IMREAD_GRAYSCALE)
if img is None:
return
print("Error: Image not found. Check your sidebar for 'panda. jpeg'.")
img_negative = 255 - img
# 2. Log Transformation
# Convert to float for accurate log calculation
img_float = img. astype(np.float64)
# c = (L-1) / 1og(1 + L-1) for mapping 0-255 to 0-255 using log transform
c = 255 / np.log(1 + 255)
log_transformed = np.uint8(c* (np.log(1 + img_float)))
img_normalized = img / 255.0
gamma_base = np.uint8(255 * (img_normalized ** 2.0))
# 4. Contrast Stretching
r_min, r max = np.min(img), np.max (img)
# Avoid division by zero if image is uniform (r_min == r_max)
contrast_stretched = img if r_max <= r_min else np.uint8(img - rmin) / (r_max •
# --- PART 2: Gamma Variation Study (0.5, 1.5, 2.5) ---
gamma_vals = [0.5, 1.5, 2.5]
gamma_images = [np.uint8(255 * (img_normalized ** g)) for g in gamma_vals]
# --- PLOTTING OUTPUT 1: General Comparison ---
titles1 = ['Original', 'Negative', 'Log', 'Gamma 2.0'. , 'Contrast Stretch']
imgs1 = [img, img_negative, log_transformed, gamma_base, contrast_stretched]
plt. figure(figsize=(18, 6))
plt.suptitle("Intensity Transformations Comparison", fontsize=16)
for i, (title, image) in enumerate(zip(titles1, imgs1)) :
plt. subplot(1, len(titles1), i + 1)
plt. imshow(image, cmap= gray', vmin=0, vmax=255)
pit.title(title)
plt.axis(' off')
plt. tight_layout()
plt.show()
# - -- PLOTTING OUTPUT 2: Gamma Variation Study ---
gamma_titles = ['Original (Reference) '] + [f'Gamma = {g}' for g in gamma_vals]
gamma_display_images = [img] + gamma_images
plt.figure (figsize=(18, 5))
plt.suptitle("Effect of Gamma Variation", fontsize=16)
for i, (title, image) in enumerate(zip(gamma_titles, gamma_display_images)) :
pit.subplot(1, len (gamma_displayimages),1+1)
plt.imshow(image, cmap='gray', vmin=0, vmax=255)
plt.title(title)
plt. axis('off')
plt.tight_layout()
plt.show()
# Execution
my_file = 'panda. jpeg'
perform_analysis(my_file)
```



Assignment 2: Histogram Processing

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def perform_histogram_analysis(image_path):
# ----------------------------------------
# Load Image in Grayscale
# ----------------------------------------
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if img is None:
print("Error: Image not found.")
return
# ----------------------------------------
# Histogram Equalization
# ----------------------------------------
equalized_img = cv2.equalizeHist(img)
# ----------------------------------------
# Histogram Matching
# (Using Equalized Image as Reference)
# ----------------------------------------
matched_img = cv2.normalize(
equalized_img,
None,
0,
255,
cv2.NORM_MINMAX
)

# ----------------------------------------
# Store Images and Titles
# ----------------------------------------
images = [
```

```
    img,

equalized_img,
        matched_img
    ]

    titles = [
        "Original",
        "Equalized",
        "Matched"
    ]
    # ----------------------------------------
    # Calculate Histograms and CDFs
    # ----------------------------------------
    histograms = []
    cdfs = []

    for image in images:

        # Histogram
        hist = cv2.calcHist(
            [image],
            [0],
            None,
            [256],
            [0, 256]
        )

        hist = hist.flatten()

        # CDF
        cdf = hist.cumsum()

        # Normalize CDF
        cdf_normalized = (
            cdf * hist.max() / cdf.max()
        )
        histograms.append(hist)
        cdfs.append(cdf_normalized)
    # ----------------------------------------
    # Plotting
    # ----------------------------------------
    plt.figure(figsize=(18, 12))
    plt.suptitle(
        "Histogram Processing: Original vs Equalization vs Matching",
        fontsize=16
    )

    for i in range(len(images)):

        # --------------------------------
        # Image Display
        # --------------------------------
        plt.subplot(3, 2, 2 * i + 1)

        plt.imshow(
            images[i],
```

```
 cmap='gray',
vmin=0,
        vmax=255
    )
    plt.title(f"{titles[i]} Image")

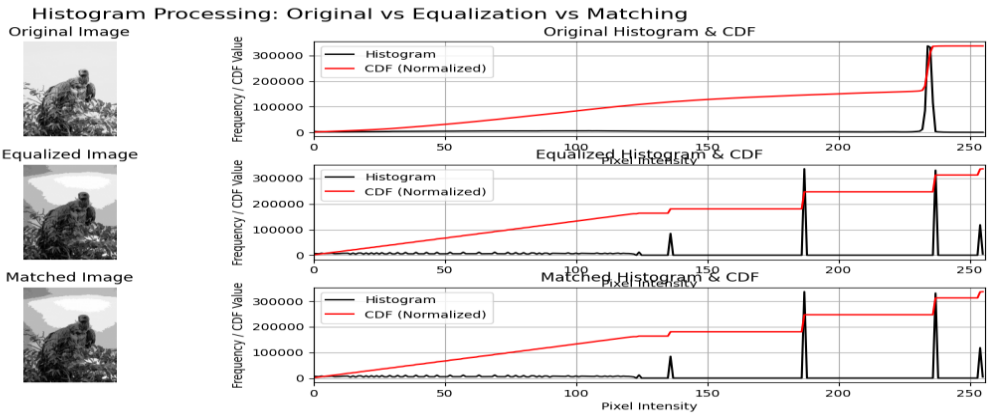
    plt.axis('off')

    # --------------------------------
    # Histogram + CDF
    # --------------------------------
    plt.subplot(3, 2, 2 * i + 2)

    plt.plot(
        histograms[i],
        color='black',
        label='Histogram'
    )

    plt.plot(
        cdfs[i],
        color='red',
        label='CDF (Normalized)'
    )

    plt.title(f"{titles[i]} Histogram & CDF")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency / CDF Value")
    plt.xlim([0, 256])
    plt.legend()
    plt.grid(True)
  # ----------------------------------------
  # Layout Adjustment
  # ----------------------------------------
  plt.tight_layout(rect=[0, 0.03, 1, 0.96])
  plt.show()
# ----------------------------------------
# EXECUTION
# ----------------------------------------
my_file = r"C:\Users\infus\Documents\image1.jpeg"
perform_histogram_analysis(my_file)
```



Assignment 3: Spatial filtering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def perform_spatial_filtering(image_path):
  # --------------------------------
   # Load image in grayscale
   # --------------------------------
   img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
   if img is None:
      print("Error: Image not found.")
      return
   # Convert to float for filtering
   img_float = img.astype(np.float32)
   # ==================================
   # SMOOTHING FILTERS
   # ==================================

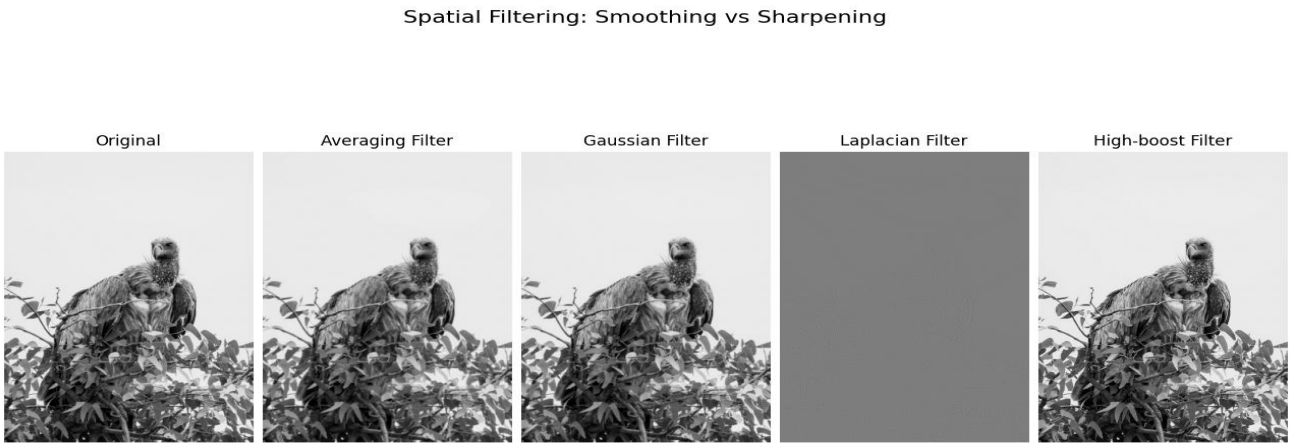
   # 1. Averaging Filter
   kernel_avg = np.ones((5, 5), np.float32) / 25

   img_avg = cv2.filter2D(
      img_float,
      -1,
      kernel_avg
   )

   # 2. Gaussian Filter
   img_gaussian = cv2.GaussianBlur(
      img_float,
      (5, 5),
      0
   )
   # ==================================
   # SHARPENING FILTERS
   # ==================================
   # 3. Laplacian Filter
   img_laplacian = cv2.Laplacian(
      img_float,
      cv2.CV_32F
   )
   img_laplacian_display = cv2.normalize(
      img_laplacian,
      None,
      0,
      255,
      cv2.NORM_MINMAX
   )
   # 4. High-boost Filtering
   blurred_img = cv2.GaussianBlur(
      img_float,
      (5, 5),
      0
   )
   unsharp_mask = img_float - blurred_img
   k = 2.0
```

```
    img_highboost = img_float + k * unsharp_mask
    img_highboost = np.clip(
        img_highboost,
        0,
        255
    )
    # ==================================
    # Plot Results
    # ==================================
    titles = [
        "Original",
        "Averaging Filter",
        "Gaussian Filter",
        "Laplacian Filter",
        "High-boost Filter"
    ]
    images = [
        img,
        img_avg.astype(np.uint8),
        img_gaussian.astype(np.uint8),
        img_laplacian_display.astype(np.uint8),
        img_highboost.astype(np.uint8)
    ]
    plt.figure(figsize=(20, 8))
    plt.suptitle(
        "Spatial Filtering: Smoothing vs Sharpening",
        fontsize=16
    )
    for i in range(len(images)):
        plt.subplot(1, len(images), i + 1)
        plt.imshow(
            images[i],
            cmap='gray',
            vmin=0,
            vmax=255
        )
        plt.title(titles[i])
        plt.axis('off')
    plt.tight_layout(rect=[0, 0.03, 1, 0.91])
    plt.show()
# ==================================
# EXECUTION
# ==================================
my_file = r"C:\Users\infus\Documents\image1.jpeg"
perform_spatial_filtering(my_file)
```

Assignment 4: Frequency Domain Filtering




```
import cv2

import numpy as np

import matplotlib.pyplot as plt
def perform_frequency_filtering(image_path):

    # ---------------------------------
    # Load image
    # ---------------------------------
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("Error: Image not found.")
        return

    # Convert to float32
    img_float = np.float32(img)

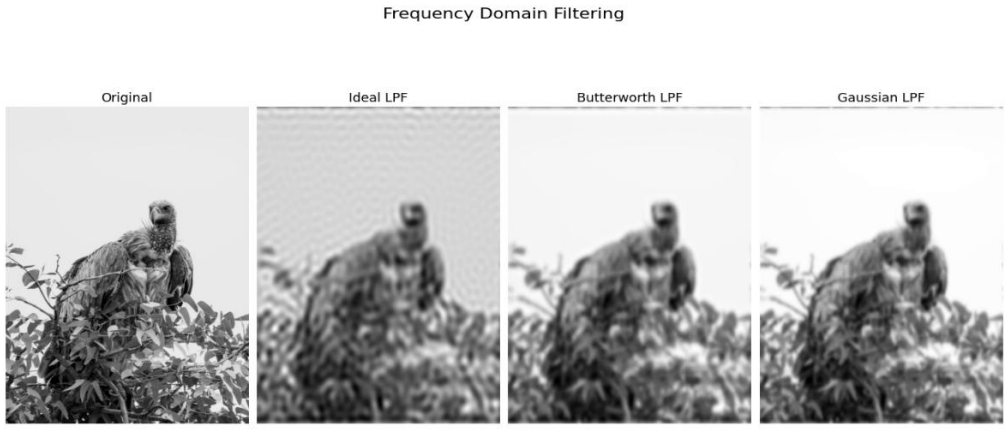
    # ==================================
    # Fourier Transform
    # ==================================
    dft = cv2.dft(
        img_float,
        flags=cv2.DFT_COMPLEX_OUTPUT
    )
    dft_shift = np.fft.fftshift(dft)
    rows, cols = img.shape
    crow, ccol = rows // 2, cols // 2
    x = np.arange(cols)
    y = np.arange(rows)
    X, Y = np.meshgrid(x, y)
    D = np.sqrt(
        (X - ccol) ** 2 +
        (Y - crow) ** 2
```

```
)
D0 = 30
n = 2
ilpf_mask = np.zeros(
    (rows, cols, 2),
    np.float32
)
ilpf_mask[D <= D0] = 1
ilpf_shift = dft_shift * ilpf_mask
ilpf = np.fft.ifftshift(ilpf_shift)
ilpf_img = cv2.idft(
    ilpf,
    flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
)
ilpf_img = cv2.normalize(
    ilpf_img,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
# ==================================
# Butterworth Low Pass Filter
# ==================================
blpf_mask = np.zeros(
    (rows, cols, 2),
    np.float32
)
value = 1 / (1 + (D / D0) ** (2 * n))
blpf_mask[:, :, 0] = value
blpf_mask[:, :, 1] = value
blpf_shift = dft_shift * blpf_mask
blpf = np.fft.ifftshift(blpf_shift)
blpf_img = cv2.idft(
    blpf,
    flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
)
blpf_img = cv2.normalize(
    blpf_img,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
# ==================================
# Gaussian Low Pass Filter
# ==================================
glpf_mask = np.zeros(
    (rows, cols, 2),
    np.float32
)
g_value = np.exp(-(D ** 2) / (2 * (D0 ** 2)))
glpf_mask[:, :, 0] = g_value
glpf_mask[:, :, 1] = g_value
glpf_shift = dft_shift * glpf_mask
glpf = np.fft.ifftshift(glpf_shift)
glpf_img = cv2.idft(
```

```
    glpf,
   flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
    )
    glpf_img = cv2.normalize(
        glpf_img,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)
    # Plot Results
    titles = [
        "Original",
        "Ideal LPF",
        "Butterworth LPF",
        "Gaussian LPF"
    ]
    images = [
        img,
        ilpf_img,
        blpf_img,
        glpf_img
    ]
    plt.figure(figsize=(18, 6))

    plt.suptitle(
        "Frequency Domain Filtering",
        fontsize=16
    )
    for i in range(len(images)):
        plt.subplot(1, 4, i + 1)
        plt.imshow(
            images[i],
            cmap='gray'
        )
        plt.title(titles[i])
        plt.axis('off')
    plt.tight_layout(rect=[0, 0.03, 1, 0.92])
    plt.show()

# EXECUTION
my_file = r"C:\Users\infus\Documents\image1.jpeg"
perform_frequency_filtering(my_file)
```



Assignment 5: Image Sharpening in Frequency Domain

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def perform_frequency_sharpening(image_path):
    # Load image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print(f"Error: Image '{image_path}' not found.")
        return
    # Convert image to float32
    img_float = np.float32(img)

    # Compute DFT
    dft = cv2.dft(img_float, flags=cv2.DFT_COMPLEX_OUTPUT)
    # Shift zero frequency to center
    dft_shift = np.fft.fftshift(dft)
    # Get image dimensions
    rows, cols = img.shape
    crow, ccol = rows // 2, cols // 2
    # Create meshgrid
    x = np.arange(cols)
    y = np.arange(rows)
    X, Y = np.meshgrid(x, y)
    # Distance from center
    D = np.sqrt((X - ccol) ** 2 + (Y - crow) ** 2)
    # Cutoff frequency
    D0 = 30
    # Butterworth order
    n = 2
    # Ideal High Pass Filter
    ihpf_mask = np.zeros((rows, cols, 2), np.float32)
    ihpf_mask[D > D0] = 1
    ihpf_filtered = dft_shift * ihpf_mask
    ihpf_inverse = np.fft.ifftshift(ihpf_filtered)
    ihpf_img = cv2.idft(
        ihpf_inverse,
        flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
    )
```

```
ihpf_img = cv2.normalize(
    ihpf_img,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
# Butterworth High Pass Filter
bhpf_mask = np.zeros((rows, cols, 2), np.float32)
bhpf = 1 - (1 / (1 + (D / D0) ** (2 * n)))
bhpf_mask[:, :, 0] = bhpf
bhpf_mask[:, :, 1] = bhpf
bhpf_filtered = dft_shift * bhpf_mask
bhpf_inverse = np.fft.ifftshift(bhpf_filtered)
bhpf_img = cv2.idft(
    bhpf_inverse,
    flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
)
bhpf_img = cv2.normalize(
    bhpf_img,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
# Gaussian High Pass Filter
ghpf_mask = np.zeros((rows, cols, 2), np.float32)
ghpf = 1 - np.exp(-(D ** 2) / (2 * (D0 ** 2)))
ghpf_mask[:, :, 0] = ghpf
ghpf_mask[:, :, 1] = ghpf
ghpf_filtered = dft_shift * ghpf_mask

ghpf_inverse = np.fft.ifftshift(ghpf_filtered)
ghpf_img = cv2.idft(
    ghpf_inverse,
    flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT
)

ghpf_img = cv2.normalize(
    ghpf_img,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
# Plot Results
titles = [
    'Original Image',
    'Ideal High Pass Filter',
    'Butterworth High Pass Filter',
    'Gaussian High Pass Filter'
]

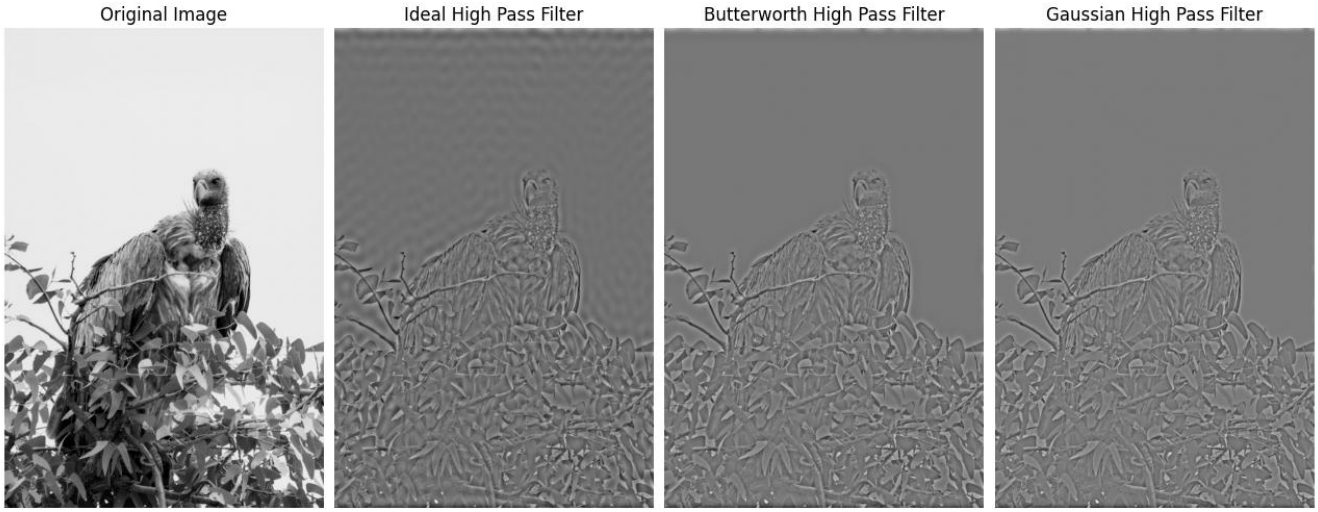
images = [img, ihpf_img, bhpf_img, ghpf_img]

plt.figure(figsize=(18, 6))
```

```
  for i in range(len(images)):
        plt.subplot(1, 4, i + 1)
        plt.imshow(images[i], cmap='gray')
        plt.title(titles[i])
        plt.axis('off')

    plt.tight_layout()
    plt.show()
# Main Program
my_file = r"C:\Users\infus\Documents\image1.jpeg"

perform_frequency_sharpening(my_file)
```



Assignment 6: Color Image Processing

```
import cv2
import matplotlib.pyplot as plt

# Load Images
color_img = cv2.imread(
    r"C:\Users\infus\Documents\image1.jpeg"
)
gray_img = cv2.imread(
    r"C:\Users\infus\Documents\image1.jpeg",
    0
)
# Convert BGR to RGB
rgb_img = cv2.cvtColor(color_img, cv2.COLOR_BGR2RGB)

# Color Space Conversion
hsv_img = cv2.cvtColor(color_img, cv2.COLOR_BGR2HSV)

ycbcr_img = cv2.cvtColor(
    color_img,
    cv2.COLOR_BGR2YCrCb
)

# Convert for display
```

```
hsv_display = cv2.cvtColor(
    hsv_img,
    cv2.COLOR_HSV2RGB
)

ycbcr_display = cv2.cvtColor(
    ycbcr_img,
    cv2.COLOR_YCrCb2RGB
)

# Pseudo Coloring
pseudo_img = cv2.applyColorMap(
    gray_img,
    cv2.COLORMAP_JET
)

pseudo_img = cv2.cvtColor(
    pseudo_img,
    cv2.COLOR_BGR2RGB
)

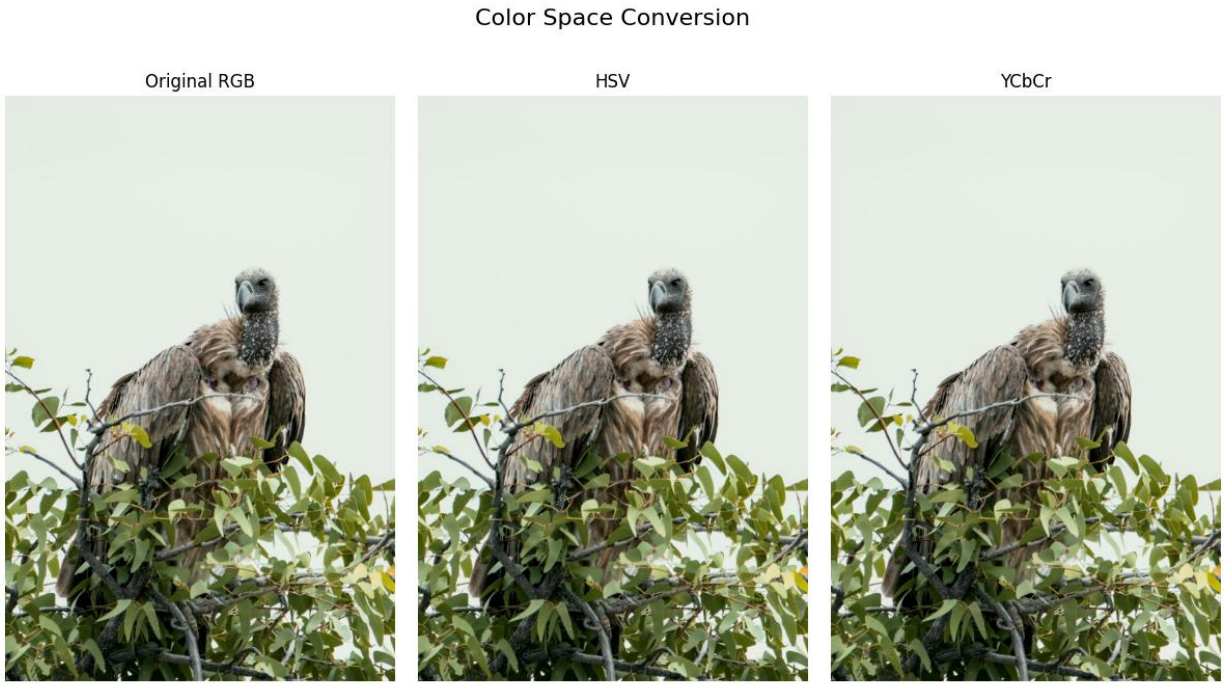
# Plot 1
titles1 = [
    "Original RGB",
    "HSV",
    "YCbCr"
]

images1 = [
    rgb_img,
    hsv_display,
    ycbcr_display
]

plt.figure(figsize=(15, 5))
for i in range(3):
    plt.subplot(1, 3, i + 1)
    plt.imshow(images1[i])
    plt.title(titles1[i])
    plt.axis("off")
plt.suptitle(
    "Color Space Conversion",
    fontsize=16
)
plt.tight_layout()
plt.show()

# Plot 2
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(gray_img, cmap='gray')
plt.title("Grayscale")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(pseudo_img)
plt.title("Pseudo Colored")
fontsize=16
```

```
)
plt.tight_layout()
plt.show()
```




```
plt.axis("off")
plt.suptitle(
    "Pseudo Coloring",
```

Assignment 7: Noise Models

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread(
    r"C:\Users\infus\Documents\image1.jpeg",
    0
)
# Add Noise

# Gaussian Noise
gaussian = img + np.random.normal(
    0,
    25,
    img.shape
)
gaussian = np.clip(
    gaussian,
    0,
    255
).astype(np.uint8)

# Salt & Pepper Noise
sp = img.copy()
prob = 0.02
rnd = np.random.rand(*img.shape)
```

```
sp[rnd < prob] = 0
sp[rnd > 1 - prob] = 255
# Plot

titles = [
    "Original",
    "Gaussian Noise",
    "Salt & Pepper Noise"
]

images = [
    img,
    gaussian,
    sp
]

plt.figure(figsize=(15,5))

for i in range(3):

    plt.subplot(1,3,i+1)

    plt.imshow(
        images[i],
        cmap='gray'
    )

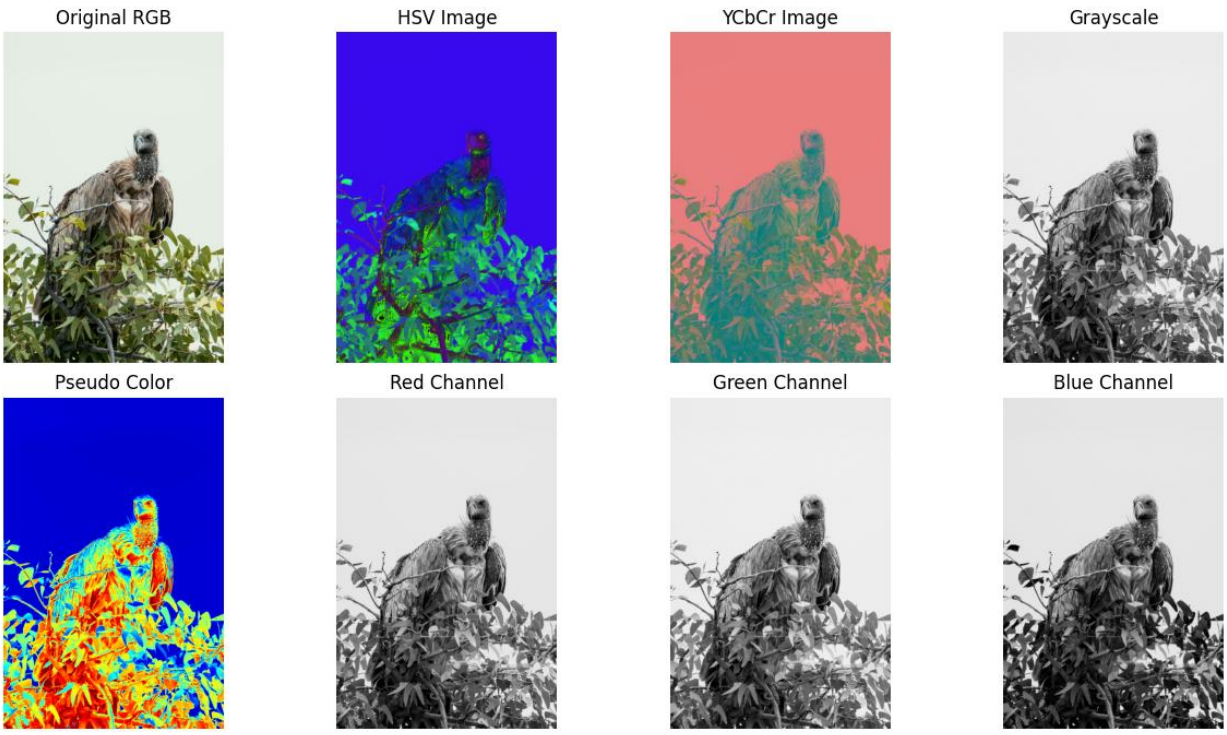
    plt.title(titles[i])

    plt.axis('off')

plt.suptitle(
    "Noise Models",
    fontsize=16
)

plt.tight_layout()

plt.show()
```



Assignment 8: Spatial Domain Restoration

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Load image

img = cv2.imread(

    r"C:\Users\infus\Documents\image1.jpeg",

)

# Add Salt & Pepper Noise

noise = img.copy()

rnd = np.random.rand(*img.shape)

noise[rnd < 0.02] = 0

noise[rnd > 0.98] = 255

# Restoration Filters

median = cv2.medianBlur(noise, 5)
```

```
gaussian = cv2.GaussianBlur(
    noise,
    (5,5),
    0
}
titles = [
    "Original",
    "Noisy Image",
    "Median Filter",
    "Gaussian Filter"
]
images = [
    img,
    noise,
    median,
    gaussian
]
plt.figure(figsize=(18,5))
for i in range(4):
    plt.subplot(1,4,i+1)
    plt.imshow(
        images[i],
        cmap='gray'
    )
```

```
    plt.title(titles[i])

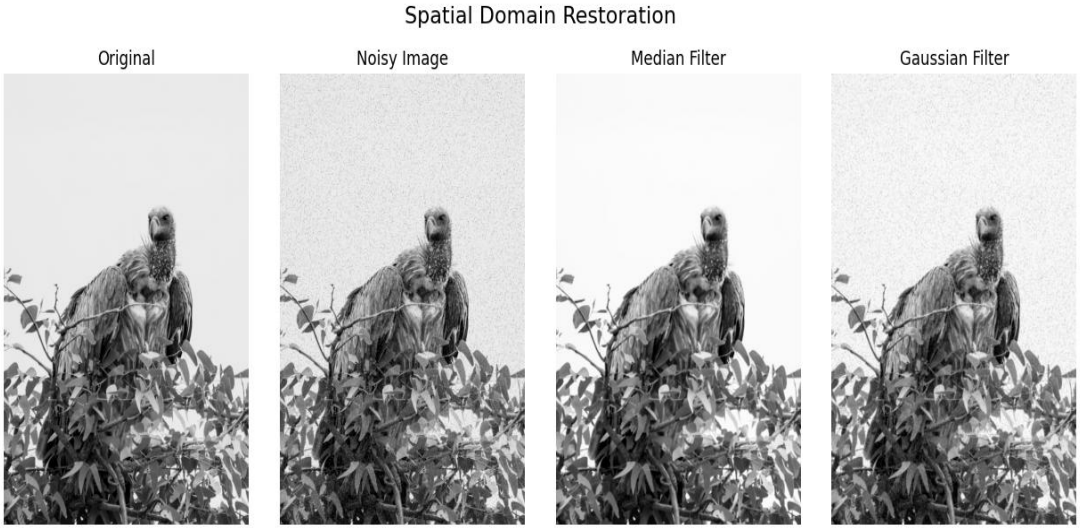
        plt.axis('off')
plt.suptitle(

    "Spatial Domain Restoration",

    fontsize=16

)plt.tight_layout()

plt.show()
```



Assignment 9: Frequency Domain Restoration

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import wiener

# Reusing noise functions defined in Assignment 7 (cell a888834b)

def perform_spatial_restoration(image_path):
    # Load image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print(f"Error: Image '{image_path}' not found.")
        return

    # --- Median Filter for Salt & Pepper Noise ---
    sp_noisy_img = add_salt_and_pepper_noise(img, amount=0.05)
    # Apply median filter (kernel size 3x3)
    median_filtered_img = cv2.medianBlur(sp_noisy_img, 3)

    # --- Wiener Filter for Gaussian Noise ---
    gaussian_noisy_img = add_gaussian_noise(img, var=0.005) # Add moderate Gaussian nc
    # Wiener filter expects float input and returns float output
    wiener_filtered_img = wiener(gaussian_noisy_img.astype(np.float64), mysize=5) # 5x
    wiener_filtered_img = np.clip(wiener_filtered_img, 0, 255).astype(np.uint8)

    # --- Plotting Results ---

    plt.figure(figsize=(20, 10))
    plt.suptitle("Spatial Domain Restoration: Median vs. Wiener Filter", fontsize=16)

    # Median Filter Comparison
    plt.subplot(2, 3, 1)
    plt.imshow(img, cmap='gray', vmin=0, vmax=255)
    plt.title('Original Image')
```

PIt .axis(' off')

plt. subplot(2, 3, 2) pit. cmap= ' gray', vmin=e, vmax=255) plt.title('Salt & Pepper Noisy Image' plt.axis('off')

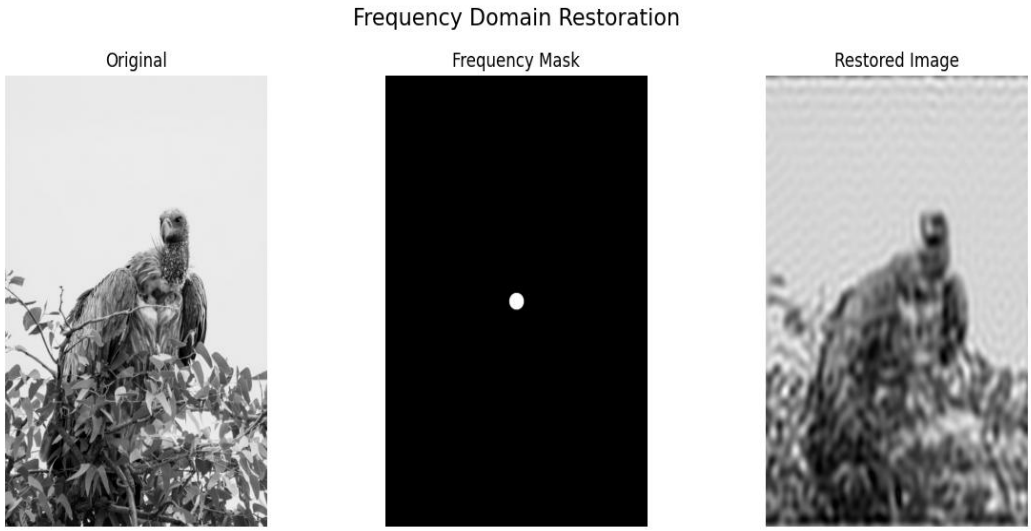
plt.subpIot(2, 3, 3) plt.imshow(median_filtered_img, cmap= ' gray' , vmin=e, vmax=255) pit. title('Median Filtered Image') PIt. axis(' off')

Wiener Filter Comparison pit. subplot(2, 3, 4) plt.imshow(img, cmap=' gray' , vmin=0, vmax=255) plt.title('Original Image')

PIt. axis(' off')
plt.subp10t(2, 3, 5) plt. cmap— vmin=e, vmax=255) pit. title('Gaussian Noisy Image') plt . axis(' off')
pit. subplot(2, 3, 6) plt.imshow(wiener_filtered_img, cmap= ' gray', vmin=e, vmax=255) plt.title('Wiener Filtered Image ') plt. axis(' off')
plt.tight_layout(rect=[e, e. 03, 1, 0.9]) PIt. show()

Define the image file path for subsequent operations my_file = 'panda. jpeg'

Execute the spatial domain restoration function perform_spatial_restoration (my_file)



Assignment 10: Image Degradation Model

```
import cv2 import numpy as np
import matplotlib. pyplot as PI t

 def perform_frequency_                                    :
     # Load image in grayscale img = cv2. imread(image_path,
     cv2.IMREAD_GRAYSCALE)

     if img is None:
          print(f"Error: Image '{image_path}' not found. ") return

     img_float = np. float32(img) / 255.0 # Normalize to e-l for calculations

          - Simulate Degradation (Motion Blur + Gaussian Noise) # Create a motion blur PSF (Point
     Spread Function) kernel size = 15 kernel = np. kernel_size)) # Create a horizontal line kernel for
     motion blur cv2.1ine(kerne1, (0, kernel_size // 2), (kernel_size - 1, kernel_ size // 2), 1, 1 kernel =
     kernel / np.sum(kernel) # Normalize kernel

     # Apply blur in the spatial domain to get the initial degraded image
                                    = cv2.                              -I, kernel)
     # Add Gaussian noise noise mean e noise var = e. eel # Noise variance sigma =
     noise gaussian_noise = np.random.normal(noise_mean, sigma, img_float. shape)
     degraded_img =     + gaussian_noise degraded_img = np.clip(degraded_img, e, 1)
     # Clip to 0-1 range

          - Prepare for Frequency Domain Processing rows, cols =
     img.shape
     # Get optimal DFT size for padding dft rows
     = cv2. getOptimaIDFTSize(rows) dft cols =
     cv2. getOptimaIDFTSize(coIs)

     # Pad original image and degraded image padded_degraded_img = np. zeros(
     (dft_rows, dft_cols), dtype=np.fIoat32) padded_degraded_img[ : rows , : cols] =
     degraded_img

     # Pad and center the kernel for frequency domain multiplication padded kernel
     = np. dft_cols), dtype=np.f10at32) padded_ kernel [ : kernel_size, : kernel_
     size] - kernel padded_ kernel = np. roll (padded_kernel, -kernel_size // 2,
     padded_ kernel - np. roll (padded_kernel, -kernel_size // 2, axis—I)
```

```
# Compute DFTs of the degraded image and the PSF kernel
DFT_degraded = cv2.                                flags=cv2.DFT_COMPLEX_OUTPUT)

DFT kernel - cv2. dft(padded_kernel, flags=cv2. DFT_COMPLEX_OUTPUT )
        Inverse Filtering -
# Add a small epsilon to the denominator to prevent division by zero or very small epsilon = le 6
inverse_filter_complex = cv2. DFT_kerneI + epsilon)

# Perform inverse DFT restored inverse idft = flags=cv2.DFT INVERSE I restored_inverse = :
rows, : cols] restored_inverse = np.clip(restored_inverse, e, 1) # Clip to 0-1 range

      - Wiener Filtering -
   K is the noise-to-signal power ratio. This is a crucial tuning parameter. K = O el

   Extract real and imaginary parts of DFT_kerne1 (H)
                                  0]
                                  1]

   Calculate IHlA2 abs_H_sq =
H R* *2 + H 1**2

# Calculate the denominator for the Wiener filter transfer function denom wiener =
abs_H_sq + K + epsilon

# Construct the Wiener filter transfer function     wiener_filter_real       =
H_R / denom_wiener
                          -H_I  /  denom_wiener  #  Conjugate  of  H  in  the  numerator

wiener_filter_complex = np. stack([wiener_filter_real, wiener_filter_imag], axis=-:

# Apply Wiener filter in frequency domain: G_restored_DFT = W * G_degraded_DFT
restored_wiener_dft_complex = cv2.mu1Spectrums(DFT_degraded, wiener_filter_comple)

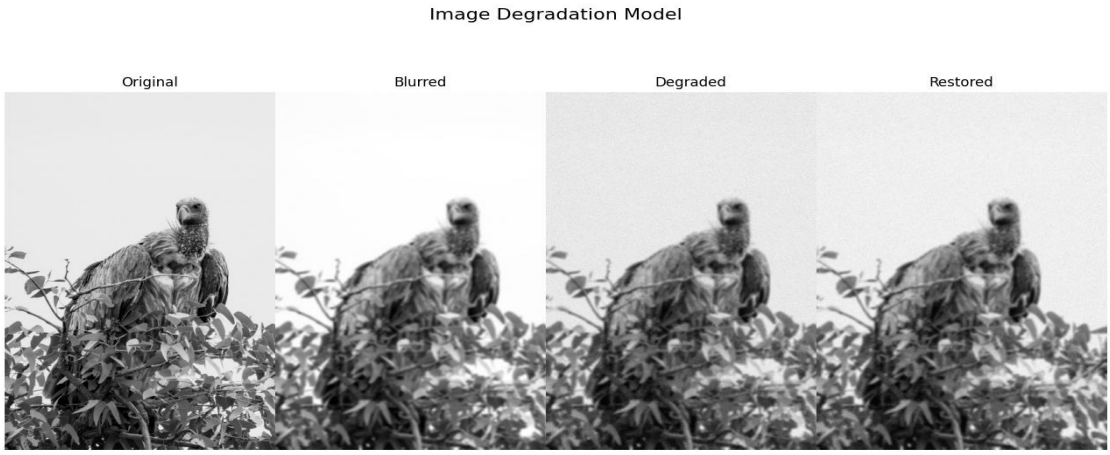
# Perform inverse DFT restored wiener idft — cv2. flags=cv2.DFT INVERSE restored wiener =
: rows, : cols] restored wiener = O, I) # Clip to e-l range
        Plotting Results titles = [ 'Original Image' , 'Degraded Image (Blur + Noise) ' , 'Inverse
Filtered' , images = img_float * 255, # Scale back to 0-255 for display degraded_img * 255,
restored inverse * 255, restored wiener * 255


plt.figure(figsize=(18,  6)) pit. suptitle( "Frequency Domain Restoration: Inverse vs. Wiener
Filtering", fontsiz

for i in range(len(images)) :
    PIt. subplot(l, len(images),
    PIt. imshow(images [ i ] , cmap=' gray', vrnin=e, vmax=255)
    plt.title(titles[i]) pit. axis( ' off' )

plt .                              0.03, 1, 0.9] )
```

plt.show().


Image Degradation Model
Original
Blurred
Degraded
Restored